

Step3: 数据库规范化与数据可视化

1 函数依赖分析与规范化

1.1 宽表结构回顾

阶段二的 `cleaned_flat_table` 包含 13 列（含代理主键 `id`），业务候选键为 `(order_id, order_item_id, review_id)`。以下分析基于该候选键。

1.2 函数依赖集

编号	函数依赖	类型
FD1	$order_id \rightarrow customer_unique_id, order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date$	部分依赖（候选键的真子集决定非主属性）
FD2	$(order_id, order_item_id) \rightarrow product_id, price, freight_value$	部分依赖（候选键的真子集决定非主属性）
FD3	$product_id \rightarrow product_category_name$	传递依赖（通过 $order_items \rightarrow product_id \rightarrow product_category_name$ ）
FD4	$(review_id, order_id) \rightarrow review_score$	部分依赖（候选键的真子集决定非主属性）

1.3 冗余与异常分析

宽表存在以下问题：

1. 数据冗余：
 - 同一客户的 `customer_unique_id` 在其所有订单明细行中重复出现
 - 同一订单的时间戳字段在该订单的每个明细×评价行中重复
 - 同一商品的 `product_category_name` 在每次出现时重复存储
2. 插入异常：无法单独添加一个新客户或新商品，必须伴随完整的订单明细和评价记录
3. 删除异常：删除某商品的最后一条订单明细，会导致该商品的类别信息丢失
4. 更新异常：修改某商品的类别名称，需要更新所有包含该商品的行，部分更新会导致数据不一致

2 概念设计

2.1 E-R 图

根据分析目标 and 数据特征，识别出 5 个实体及其联系：

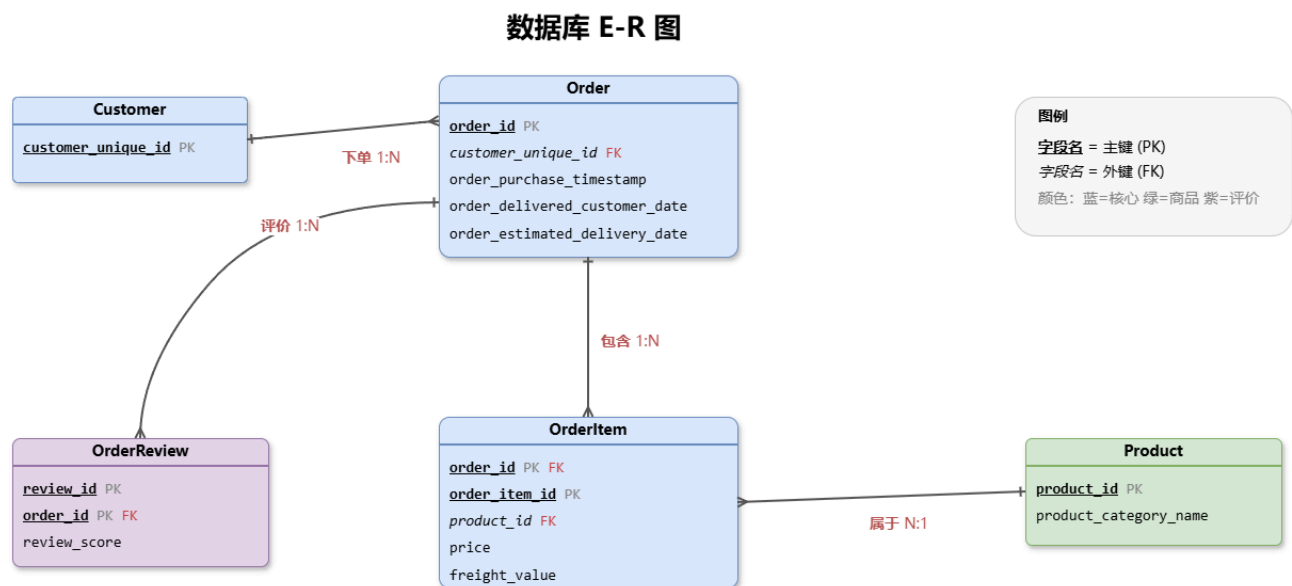
实体与属性：

实体	主键	其他属性
Customer	customer_unique_id	—
Order	order_id	order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date
OrderItem	(order_id, order_item_id)	price, freight_value
Product	product_id	product_category_name
OrderReview	(review_id, order_id)	review_score

联系与类型：

联系	类型	说明
Customer → Order	1:N	一个客户可下多个订单
Order → OrderItem	1:N	一个订单包含多个商品项
Product → OrderItem	1:N	一个商品可出现在多个订单项中
Order → OrderReview	1:N	一个订单可有多条评价

E-R图：



3 逻辑结构设计与规范化

3.1 E-R 图转关系模式

将 E-R 图中的 5 个实体直接转换为 5 个关系模式：

- **customers**(customer_unique_id)
- **orders**(order_id, *customer_unique_id*, order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date)
- **products**(product_id, product_category_name)
- **order_items**(order_id, order_item_id, *product_id*, price, freight_value)
- **order_reviews**(review_id, *order_id*, review_score)

其中下划线标注主键，斜体为外键。

3.2 3NF 分解过程

从宽表 R(order_id, order_item_id, review_id, customer_unique_id, order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date, product_id, product_category_name, price, freight_value, review_score) 出发：

第一步：消除部分依赖（达到 2NF）

- FD1: order_id → customer_unique_id, 时间戳×3，候选键的真子集决定非主属性 → 拆出 **orders** 表
- FD2: (order_id, order_item_id) → product_id, price, freight_value，候选键的真子集决定非主属性 → 拆出 **order_items** 表
- FD4: (review_id, order_id) → review_score，候选键的真子集决定非主属性 → 拆出 **order_reviews** 表

第二步：消除传递依赖（达到 3NF）

- FD3: 在 order_items 中，product_id → product_category_name 构成传递依赖（order_items 的主键 → product_id → product_category_name）→ 拆出 **products** 表
- 同时提取 **customers** 表作为 orders 外键的参照实体

分解结果：5 张表均满足 3NF，每个非主属性完全依赖于主键且不存在传递依赖。

3.3 参照完整性

外键	所在表	参照表
orders.customer_unique_id	orders	customers
order_items.order_id	order_items	orders
order_items.product_id	order_items	products
order_reviews.order_id	order_reviews	orders

4 架构迁移

4.1 3NF 建表脚本

对应 DDL 脚本为 `schema3.sql`，在 `proj1_3nf` schema 下创建 5 张规范化表：

```
1 CREATE SCHEMA IF NOT EXISTS proj1_3nf;
2
3 CREATE TABLE customers (
4     customer_unique_id VARCHAR(32) PRIMARY KEY
5 );
6
7 CREATE TABLE products (
8     product_id VARCHAR(32) PRIMARY KEY,
9     product_category_name VARCHAR(100) NOT NULL DEFAULT 'Unknown'
10 );
11
12 CREATE TABLE orders (
13     order_id VARCHAR(32) PRIMARY KEY,
14     customer_unique_id VARCHAR(32) NOT NULL,
15     order_purchase_timestamp TIMESTAMP NOT NULL,
```

```

16     order_delivered_customer_date TIMESTAMP NOT NULL,
17     order_estimated_delivery_date TIMESTAMP NOT NULL,
18     FOREIGN KEY (customer_unique_id) REFERENCES customers(customer_unique_id)
19 );
20
21 CREATE TABLE order_items (
22     order_id VARCHAR(32) NOT NULL,
23     order_item_id INTEGER NOT NULL CHECK (order_item_id > 0),
24     product_id VARCHAR(32) NOT NULL,
25     price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
26     freight_value NUMERIC(10,2) NOT NULL CHECK (freight_value >= 0),
27     PRIMARY KEY (order_id, order_item_id),
28     FOREIGN KEY (order_id) REFERENCES orders(order_id),
29     FOREIGN KEY (product_id) REFERENCES products(product_id)
30 );
31
32 CREATE TABLE order_reviews (
33     review_id VARCHAR(32) NOT NULL,
34     order_id VARCHAR(32) NOT NULL,
35     review_score SMALLINT NOT NULL CHECK (review_score BETWEEN 1 AND 5),
36     PRIMARY KEY (review_id, order_id),
37     FOREIGN KEY (order_id) REFERENCES orders(order_id)
38 );

```

4.2 数据迁移

迁移脚本 `migrate_to_3nf.sql` 通过 `CREATE TEMP VIEW` 引用阶段二宽表，使用 `SELECT DISTINCT` 逐表提取数据：

目标表	迁移行数	说明
customers	92,753	DISTINCT customer_unique_id
products	32,070	DISTINCT product_id + category
orders	95,830	DISTINCT order_id + 时间戳
order_items	109,369	DISTINCT (order_id, order_item_id) + price/freight
order_reviews	96,359	DISTINCT (review_id, order_id) + score

4.3 迁移验证

执行 `validate_3nf.sql` 验证结果：

- 所有主键重复检查：0（无重复）
- 所有外键孤儿检查：0（无孤儿记录）
- 源宽表与目标表的粒度校验：全部匹配

```

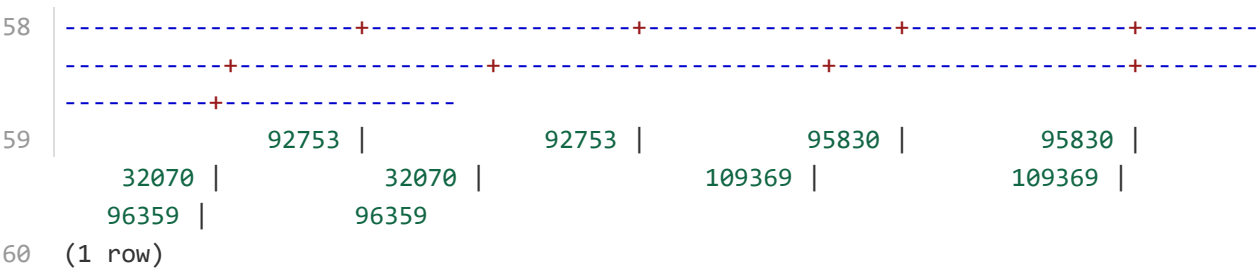
1 DROP VIEW
2 CREATE VIEW
3     table_name | row_count
4 -----+-----
5 customers    |    92753
6 products     |    32070
7 orders       |    95830
8 order_items  |   109369

```

```

9  order_reviews |      96359
10 (5 rows)
11
12 duplicated_customer_keys
13 -----
14                               0
15 (1 row)
16
17 duplicated_product_keys
18 -----
19                               0
20 (1 row)
21
22 duplicated_order_keys
23 -----
24                               0
25 (1 row)
26
27 duplicated_order_item_keys
28 -----
29                               0
30 (1 row)
31
32 duplicated_review_keys
33 -----
34                               0
35 (1 row)
36
37 orphan_orders
38 -----
39                               0
40 (1 row)
41
42 orphan_order_items_by_order
43 -----
44                               0
45 (1 row)
46
47 orphan_order_items_by_product
48 -----
49                               0
50 (1 row)
51
52 orphan_reviews
53 -----
54                               0
55 (1 row)
56
57 expected_customers | actual_customers | expected_orders | actual_orders |
   expected_products | actual_products | expected_order_items | actual_order_items |
   expected_reviews | actual_reviews

```



5 数据可视化

使用 Streamlit 框架构建数据分析看板（`project1_app.py`），通过 `psycopg2` 连接 openGauss 数据库，实时从 3NF 表中查询数据。

5.1 核心指标概览

使用 `st.metric` 组件展示三个核心指标：

指标	SQL	数据来源
总订单数	COUNT(*) FROM orders	orders 表
总客户数	COUNT(*) FROM customers	customers 表
平均评分	AVG(review_score) FROM order_reviews	order_reviews 表

5.2 分析目标 1：物流时效与客户满意度

将订单按实际送达时间是否晚于预计送达时间分为"逾期送达"和"准时送达"两组，计算各组的平均评分，以柱状图展示对比。

5.3 分析目标 2：品类消费与复购行为差异

筛选家居家装类（`moveis_decoracao` 等 6 个类别）和美妆个护类（`beleza_saude`、`perfumaria`）客户，分别计算平均订单金额和复购率（下单 ≥ 2 次的客户占比），以并排柱状图展示。

5.4 分析目标 3：2017 年季度运费成本趋势

计算 2017 年各季度订单的平均运费占商品价格比例，以柱状图展示 Q1-Q4 的趋势变化。

5.5 原始数据抽样

通过 4 表 JOIN（orders + order_items + products + order_reviews）展示前 50 条规范化后的数据样本。

运行方式：

```
1 | streamlit run project1_app.py
```

5.6 结果展示

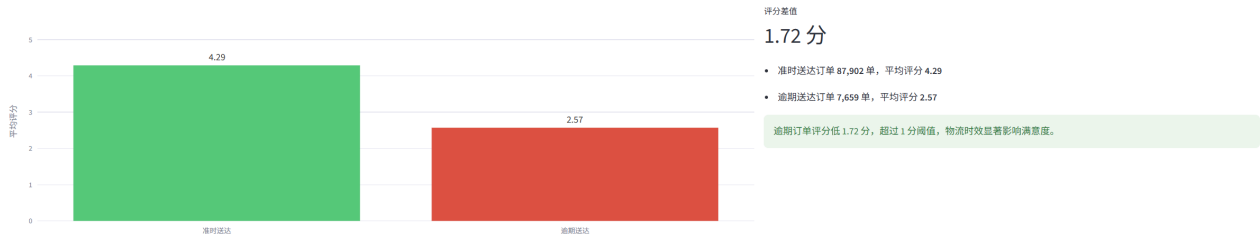
Olist Brazilian E-Commerce 数据分析看板

核心指标概览

总订单数	总客户数	平均评分
95,830	92,753	4.16

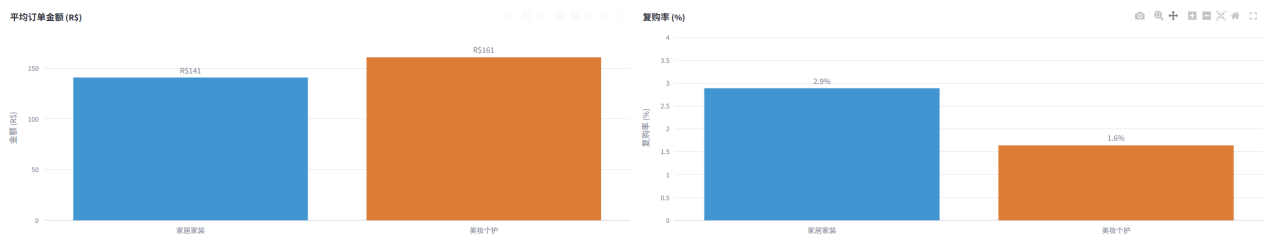
分析目标 1：物流时效与客户满意度

2017-2018 年已送达订单：逾期送达 vs 准时送达的平均评分对比



分析目标2：品类消费与复购行为差异

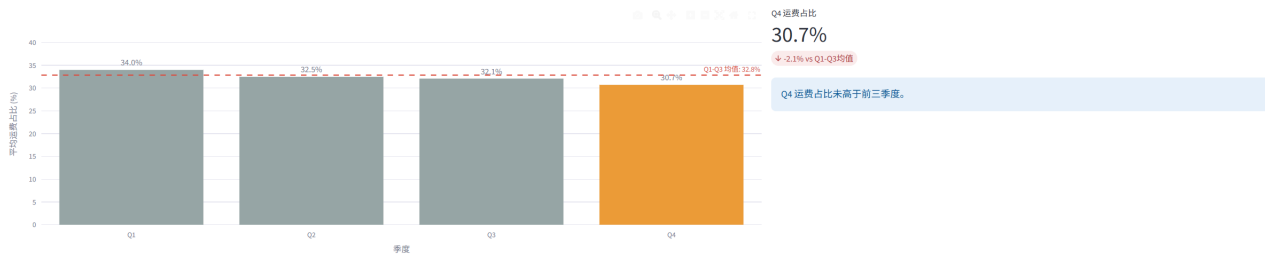
2017-2018 年家居家装类 vs 美妆个护类客户的平均订单金额与复购率



- 家居家装类客户 21,517 人，美妆个护类客户 11,409 人

分析目标 3：2017 年季度运费成本趋势

2017 年各季度订单的平均运费占商品价格比例, 检验 Q_4 是否显著高于前三季度



原始数据抽样

3NF 规范化后的数据样本 (4 表 JOIN, 前 50 行)

	order_id	customer_unique_id	order_purchase_timestamp	order_item_id	product_category_name	price	freight_value	review_score	order_delivered_customer_date	order_estimated_delivery_date
0	00010242f8c5a61ba2d7f92cb16214	871766c585e6b3f6ecce0f9888023cb	2017-09-13 08:59:02	1	cool_stuff	58.9	13.29	5	2017-09-23 23:43:48	2017-09-29 00:00:00
1	00018f7702b032c5f519d07a144b0d3	eb286c7f8c67b3846055d0fba35d051	2017-04-26 10:53:06	1	pet_shop	239.9	19.93	4	2017-05-12 16:04:24	2017-05-15 00:00:00
2	00022fec39824f6e3719d07a144b0d3	3818d81c6709e39d0b2738ald3a2474	2018-01-14 14:33:31	1	moveis_decoracao	19.9	17.87	5	2018-01-22 13:19:16	2018-02-05 00:00:00
3	00024cbcd9f46da1e913b1308114c75	a96b1436c6c08b2c2d6edfdba074622	2018-08-08 10:00:35	1	perfumaria	12.99	12.74	4	2018-08-14 13:32:39	2018-08-20 00:00:00
4	00042626f59d7ce9f6db46e55b409d	640579fb704c1d4e9f1b2d67d446e483c5	2017-02-04 13:57:51	1	ferramentas_jardim	199.9	18.19	5	2017-03-01 16:42:31	2017-03-17 00:00:00
5	00048c3ae77f656bb12d043d634c1ea	85c33d128bee5b4ce6e02c491bf385	2017-05-15 21:42:34	1	utilidades_domesticas	21.9	12.69	4	2017-05-22 13:44:35	2017-06-06 00:00:00
6	00054e6431b9d47675808b9cb319f0a32	635df9ac1680f03288e72ada3a1035803	2017-12-10 11:53:48	1	telefonias	19.9	11.85	4	2017-12-18 22:03:38	2018-01-04 00:00:00
7	000576e39319847cb9d288c5617f6e	fda4476abb6307abc3415b76a6d26526	2018-07-14 12:08:27	1	ferramentas_jardim	810	70.65	5	2018-07-09 14:04:07	2018-07-25 00:00:00
8	00055a1a1728f78d586c20b08904576c	639d2324f5517f69d0c3d6e5664f6c	2018-03-19 18:40:33	1	beleza_saude	145.95	11.75	1	2018-03-29 18:17:31	2018-03-29 00:00:00
9	000550442cb953d4cd1d2169232495	0792c14380992a5d53348906c3d0ee6f	2018-07-02 13:59:39	1	livros_tecnicos	53.99	11.4	4	2018-07-07 17:28:31	2018-07-23 00:00:00